Car Maintenance and Repair Center Management System

Project Report

Advanced Programming 2
Spring 2025

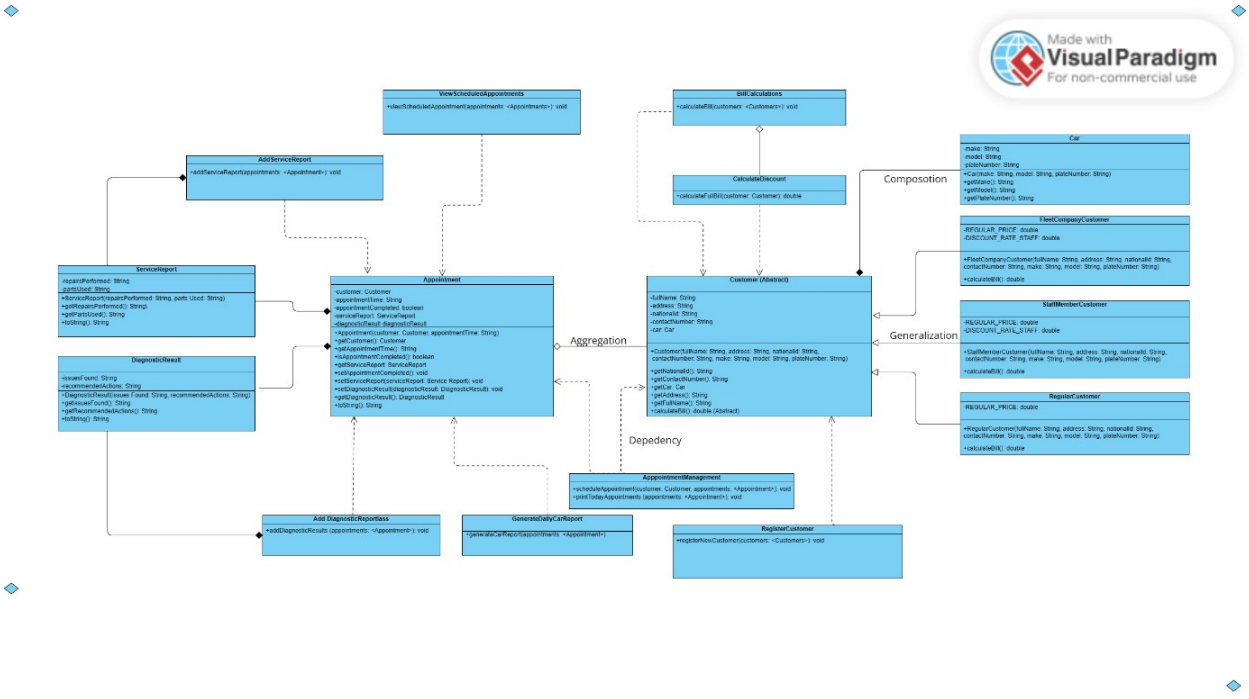
Contents

1. Class Diagram and Class Relationships....3
2. UML Diagram with Design Patterns....4
3. Test Cases....5
4. Design Patterns....6
5. Clean Code, SOLID Principles, and Design Patterns....7
6. Modifications for Testing....8
7. Automated Testing....8
References....11

1. Class Diagram and Class Relationships

In my design, I created 16 classes and applied the SOLID principles that were required, such as single responsibility, open-closed, and Liskov substitution. All clean code guidelines were also applied effectively in my design.

UML Class Diagram



Relationships between the classes

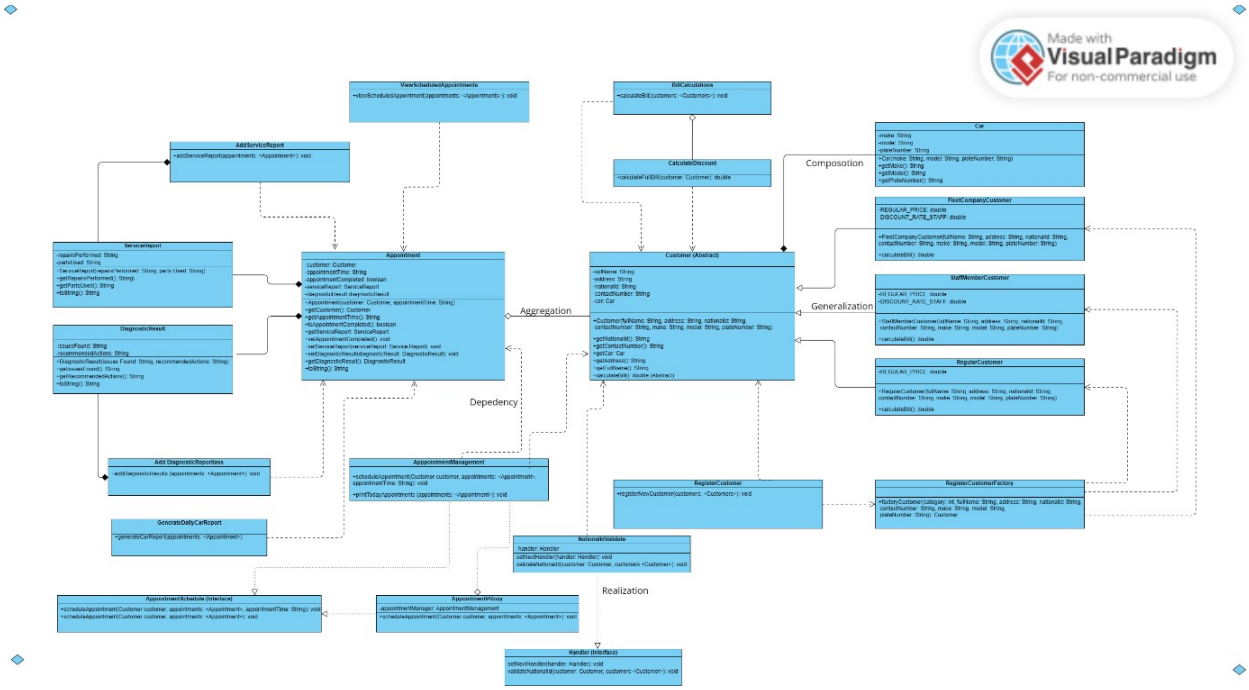
- **Customer – Appointment (Aggregation):** The relationship is aggregation because the Appointment class holds a Customer object, and in my design a customer can exist independently of an appointment. A customer can be registered without scheduling an appointment.
- **Customer – (FleetCompanyCustomer, StaffMemberCustomer, RegularCustomer) (Generalization):** These three classes inherit the fields of the Customer class. They are subclasses of Customer, which is the superclass.
- **Customer – Car (Composition):** The Customer class holds a Car object, and a car cannot exist independently of a customer. The whole system is about car maintenance, so when a customer is created, a car is created with it.
- **Customer – (CalculateDiscount, BillCalculations, RegisterCustomer, AppointmentManagement) (Dependency):** These four classes use a Customer object as a parameter in their methods, so they rely on the Customer class to perform their operations.
- **Appointment – (ServiceReport, DiagnosticReport) (Composition):** These two classes are part of Appointment, and a service report or diagnostic report cannot exist without an appointment. If an Appointment is destroyed, its ServiceReport and DiagnosticReport are destroyed with it.
- **Appointment – (AddDiagnosticReport, AddServiceReport, GenerateDailyReport, AppointmentManagement, ViewScheduledAppointments) (Dependency):** These classes use an Appointment object as a parameter in their methods.

- **ServiceReport – AddServiceReport (Composition):** AddServiceReport holds a ServiceReport object, and a ServiceReport cannot exist independently of AddServiceReport, because a service report can only be added through AddServiceReport.
- **DiagnosticReport – AddDiagnosticReport (Composition):** AddDiagnosticReport holds a DiagnosticReport object, and a DiagnosticReport cannot exist independently of AddDiagnosticReport, because a diagnostic report can only be added through AddDiagnosticReport.
- **CalculateDiscount – BillCalculations (Aggregation):** BillCalculations holds a CalculateDiscount object, and CalculateDiscount can exist independently.

2. UML Diagram with Design Patterns

In my design, I used design patterns from the creational, structural, and behavioural categories: Factory, Proxy, and Chain of Responsibility.

UML Class Diagram with Design Patterns



Relationships added by the design patterns

- **RegisterCustomerFactory – (FleetCompanyCustomer, StaffMemberCustomer, RegularCustomer) (Dependency):** RegisterCustomerFactory depends on these three classes to return the appropriate object.
- **RegisterCustomerFactory – RegisterCustomer (Dependency):** Although a RegisterCustomerFactory object is created inside a RegisterCustomer method, RegisterCustomer depends on RegisterCustomerFactory to perform its operation, so dependency is the most correct relationship.
- **AppointmentSchedular – (AppointmentProxy, AppointmentManagement) (Realization):** These two classes implement the AppointmentSchedular interface.
- **AppointmentProxy – AppointmentManagement (Aggregation):** AppointmentProxy holds an AppointmentManagement object, and if the AppointmentProxy is destroyed, AppointmentManagement still exists.

- **NationalIdValidate – Handler (Realization):** NationalIdValidate implements the Handler interface.
- **NationalIdValidate – Customer (Dependency):** A method in NationalIdValidate takes a Customer object as a parameter, so NationalIdValidate depends on Customer to perform its operation.

3. Test Cases

Test setup

Before each test, two empty lists are created:

```
ArrayList<Customer> customers = new ArrayList<>();
ArrayList<Appointment> appointments = new ArrayList<>();
```

A customer is added as soon as the test starts, and every test uses a different customer with a different national ID.

- **addServiceReport():** This functionality is one of the mechanic's responsibilities. It adds a service report to an appointment based on an appointment time.

Test Case	Test Description	Input	Expected Output

TestOneAS	Input a correct appointment time.	`RegularCustomer("Moe", "Amman", "200", "0799", "Ford", "2010", "20-94680");`
		`Appointment(customer, "9:00 AM")`
		`addServiceReport(appointments, "9:00 AM")`
			Service Report successfully added

TestTwoAS	Input an appointment time after 5:00 PM.	`RegularCustomer("motasem", "Amman", "201", "0799434", "Toyota", "2013", "20-5435");`
		`Appointment(customer, "9:00 PM")`
		`addServiceReport(appointments, "8:00 PM")`
			Please enter a valid appointment time!

TestThreeAS	Input a wrong appointment time.	`RegularCustomer("Laith", "Amman", "202", "07967", "Chevrolet", "2010", "20-5354")`
		`Appointment(customer, "11:00 AM")`
		`addServiceReport(appointments, "3:00 PM")`
			Appointment is already completed!

- **addDiagnosticResults():** This functionality is one of the technician's responsibilities. It adds a diagnostic report to an appointment based on an appointment time.

I used equivalence partitioning for these test cases.

Test Case	Test Description	Input	Expected Output

TestOneAD	Input a correct appointment time.	`RegularCustomer("Salem", "Amman", "203", "079559", "Ford", "2013", "20-6756")`
		`Appointment(customer, "9:00 AM")`
		`addDiagnosticResults(appointments, "9:00 AM")`
			Diagnostic Report successfully added

TestTwoAD	Input an appointment time after 5:00 PM.	`RegularCustomer("Ayman", "Amman", "204", "079910", "Toyota", "2012", "20-94680")`
		`Appointment(customer, "9:00 AM")`
		`addDiagnosticResults(appointments, "7:00 PM")`
			Please enter a valid appointment time!

TestThreeAD	Input an appointment time before 9:00 AM.	`("Rashed", "Amman", "205", "079910", "Ford", "2005", "20-4566")`
		`Appointment(customer, "9:00 AM")`
		`addDiagnosticResults(appointments, "8:00 AM")`
			Please enter a valid appointment time!

- **calculateBill():** This function is one of the front desk staff's responsibilities. It calculates the bill for a customer based on the national ID.

Test Case	Test Description	Input	Expected Output
TestOneS	Input a valid national ID.	`RegularCustomer("Mahmoud", "Amman", "206", "0793449", "Mercedess", "2012", "20-24434")` `calculateBill(customers, "200")`	Full Bill: $50.0
TestTwoS	Input an invalid national ID.	`RegularCustomer("Mohammad", "Amman", "207", "07998221", "BMW", "2018", "20-65466")` `calculateBill(customers, "100")`	Customer Not Found
TestThreeS	Input a valid national ID and determine whether the returned price is correct based on the customer type.	`FleetCompanyCustomer("Hamad", "Amman", "230", "0778999", "Kia", "2019", "20-6546")` `calculateBill(customers, "230")`	Full Bill: $35.0
TestFourS	Input a valid national ID and determine whether the returned price is correct based on the customer type.	`StaffMembersCustomer("Ayham", "Amman", "240", "079765", "Huynday", "2009", "20-5453")` `calculateBill(customers, "250")`	Full Bill: $25.0

4. Design Patterns

Design patterns are divided into three categories: creational, structural, and behavioural. In my code, I used one design pattern from each category, as described below.

- **Factory (creational):** I used the Factory pattern in the customer registration process. When a customer is registered, I must determine the customer type (FleetCompanyCustomer, StaffMemberCustomer, or RegularCustomer) and create the correct object. I moved this responsibility out of the RegisterCustomer class into a new class called RegisterCustomerFactory, which takes the user's choice and returns the correct customer type. This kept the registration code easier to maintain and understand, and it gave each class a single responsibility. If I want to add a new customer type, I only need to update RegisterCustomerFactory and not RegisterCustomer, which makes the code easier to maintain and extend. This approach also hides object creation from the client.
- **Proxy (structural):** I used the Proxy pattern to control access to appointment scheduling. Instead of interacting directly with the AppointmentManagement class, I created the AppointmentProxy class, which checks that the appointment time entered is valid and between 9:00 AM and 5:00 PM before forwarding the scheduling request to AppointmentManagement. This kept the logic clean and made the system more secure and flexible. The proxy lets me perform the time validation before the request reaches the original AppointmentManagement method, which prevents appointments from being scheduled outside working hours.
- **Chain of Responsibility (behavioural):** Since a national ID is unique, I used Chain of Responsibility during customer registration to check whether a national ID already belongs to a customer. I used a single chain. If the national ID already exists, the new customer is not added to the ArrayList, an error message is printed, and the chain stops. If the national ID does not exist, the new customer is added to the ArrayList. I used the chain in the RegisterCustomer class, where the customer being registered and the customer list are passed to the chain as parameters. This pattern allowed me to validate the national ID while

keeping the ability to add more validation steps, each in its own class, which keeps the code organised. It also prevents duplicate national IDs in the system.

5. Clean Code, SOLID Principles, and Design Patterns

To make sure that my system code is effective, I applied the SOLID principles, clean code techniques, and design patterns.

Clean Code Techniques

In my code, I tried to follow all of the clean code rules. I used meaningful, pronounceable, and searchable variable and method names, so anyone who reads my code can easily understand what they are about. For example, it is obvious that the appointmentTime variable in the Appointment class represents the time of an appointment. I also made each function do only one thing, which keeps the functions cleaner. For example, the registerNewCustomer method in the RegisterCustomer class has only one responsibility, which is registering the customer in the system. The method names also say what the method does. For example, addDiagnosticResults clearly adds a diagnostic result to a customer's appointment.

Following these techniques helped me a lot in keeping track of everything while building a large application like the Car Maintenance and Repair Center Management System. Without them, I would have been lost among the many classes. Anyone who reads my code can also easily understand what each class, variable, and method is about, and what the whole system does.

SOLID Principles

I applied several of the SOLID principles, which helped me build maintainable, flexible, scalable, and well-organised code.

- **Single Responsibility:** Every component should have only one responsibility and one reason to change. I applied this principle throughout the code, so every class has only one responsibility and one reason to change. For example, the Appointment class contains only components related to appointments, such as the customer, appointment time, service report, and diagnostic report. All of these are related to each other through the Appointment class, which gives it high cohesion. Following this principle made the code smaller and better organised. It also means that classes with one responsibility need fewer test cases, and it lowers coupling.

  It is worth mentioning that I have a minor violation of the interface segregation principle in my implementation of the Proxy pattern. I had to overload a method in the same interface by declaring two methods with different parameters, because the proxy class needs one method and the original class needs the other. As a result, the proxy class and the original class each override a method that they do not use.

- **Open-Closed:** This principle states that a system should be open for extension and closed for modification, so a new feature can be added without modifying the existing system. I applied it to calculating the bill for each customer type. Before applying this principle, I used the instanceof keyword in the BillCalculations class to determine the customer type and calculate the bill. After applying it, the code became open for extension and closed for modification. For example, if a new customer type is added, nothing in my code needs to be modified, and I only need to add a class for that customer type. This approach improved extensibility, maintainability, and flexibility, as the system can easily accept new features without affecting the existing code. Applying the open-closed principle also indirectly applied dependency inversion.

- **Liskov Substitution:** Objects of a parent class should be replaceable by objects of its child classes. I applied this principle to the Customer class and its subclasses FleetCompanyCustomer, StaffMemberCustomer, and RegularCustomer. Customer has an abstract method called calculateBill, and each of the three subclasses overrides it with a different implementation. This ensures that the child classes can be substituted without affecting the behaviour of the program.

Design Patterns

I used one design pattern from each category: Factory from the creational category, Proxy from the structural category, and Chain of Responsibility from the behavioural category.

- **Factory:** As described in Section 4, I used the Factory pattern to decide which type of Customer to create during registration, based on the input data. It was implemented effectively because it cleanly separates the registration process into two parts, one for the registration data and another for returning the appropriate object. It also loosely follows the open-closed principle, since I can add a new customer type by editing only the factory code and not the existing registration code. In conclusion, it keeps the whole registration process clean.
- **Proxy:** As described in Section 4, I used the Proxy pattern to handle appointment scheduling. It acts as a shield for the actual scheduling process: before forwarding a scheduling request, the proxy checks that the appointment time is within working hours. It was implemented effectively because it separates the time validation from the scheduling logic, which improves the security of the code, as appointments cannot be scheduled outside working hours. It also allows for future extension, because a new validation can be added without editing the scheduling logic, which makes the code more scalable.
- **Chain of Responsibility:** As described in Section 4, I implemented Chain of Responsibility as a single chain that validates that the national ID is not duplicated during customer registration, which prevents a duplicate national ID from being registered in the system. It was implemented effectively because each validation is independent, which makes the validation steps scalable, so more chains can be added easily. It also made the process more organised.

6. Modifications for Testing

I made minor modifications to the functions that are tested, in a way that does not affect the original logic. The functions did not work for testing before because their return type was void, so I could not use them in assertEquals. Instead of printing the message, I made each function return the value that was printed. I changed the return type of the methods, removed the Scanner, and made each method accept the value that was previously read from the Scanner, so the methods can be tested. The core functionality is the same before and after these modifications.

7. Automated Testing

a) Methods of Automated Testing

Testing is the process of discovering all potential issues in a program before deploying the system. Automated testing means testing the program without a developer performing the tests manually. The methods of implementing automated testing are:

- **Unit Testing:** A testing method that tests individual units of the system, such as a single object or function. The unit is the smallest part of an application that can be tested.

- **Component Testing:** A testing method that tests a collection of individual units that form a composite component, such as how a collection of functions works together.
- **System Testing:** A testing method that tests the whole system, such as how some or all of the components in a system work together as a whole.
- **Integration Testing:** A testing method that tests different components or systems and makes sure that they work correctly with each other.

b) Developer-Produced and Vendor-Provided Automated Testing

Before discussing the differences, I will define both terms.

- **Developer-produced automated testing:** Testing tools that are created by developers to perform automated testing.
- **Vendor-provided automated testing:** Testing tools that are provided by vendors to perform automated testing.

They differ in terms of:

- **Origin:** Developer testing tools are built to fit the developer's needs, while vendor testing tools are built to fit a wide range of users' needs.
- **Customisation:** Developer testing tools are more customisable than vendor testing tools, because they are built by an individual developer or a team of developers who have full control over them. Vendor testing tools are less customisable because they are built for a large number of users.
- **Support:** Developer testing tools do not have many support options, as the developers must resolve issues themselves, while vendor testing tools come with official support teams to resolve issues.
- **Integration:** Developer testing tools fit easily into the program because they are made for that specific program, while vendor testing tools may be harder to integrate because they are made to cover a wide range of programs.

c) Benefits and Drawbacks of Each Method

Unit Testing

Benefits

1. Breaks the system into parts so that only one specific unit is tested at a time, which makes it easier to update the code when an issue is found.
2. Helps to identify issues early, which allows them to be fixed early.
3. Finds issues before they affect other parts of the code.
4. Makes determining and fixing issues easier, as we work on a small part.
5. Lowers the cost of a project by finding issues early and avoiding late changes.

Drawbacks

1. Requires more testing code, which can increase the time and cost of the project.
2. Requires a lot of testing, as a project has many units.
3. Does not test how the units work together.

Example from my application: A unit test for the Appointment class could test isAppointmentCompleted() to check whether it correctly updates the appointment status from false to true.

Component Testing

Benefits

1. Identifies issues early.
2. Makes error tracing and fixing simple.
3. Allows better code organisation, which makes everything easier.
4. Gives more coverage than unit testing.
5. Focuses on a specific functionality.

Drawbacks

1. Does not test how different components work together.
2. If the code changes, the tests might need to be updated, which takes more time.
3. Does not test complex workflows.

Example from my application: A component could be the BillCalculations class, and to test it, we could test the logic of calculating the customer's bill.

System Testing

Benefits

1. Makes sure that the entire system meets all requirements by testing the system as a whole.
2. Detects issues in how components work together.
3. Tests the performance, security, and compatibility aspects of the system.
4. Shows that the system is ready before deployment.

Drawbacks

1. Takes a lot of time, because there is a lot to test.
2. Takes more resources and may need complex test environments.
3. If major issues are found, the deployment of the system is delayed.

Example from my application: We could write a scenario to test the whole system, for example: "the front desk registers a customer, schedules an appointment, and calculates the customer's bill based on the customer type."

Integration Testing

Benefits

1. Tests the interaction between sections and makes sure that they work correctly together.
2. Minimises the errors in the final product.
3. Improves communication between development teams working on different sections.

Drawbacks

1. Takes a lot of time and cost, as it needs different testing environments and there are many interactions to test.
2. If there are dependencies between sections, it may be hard to determine when and how to perform the integration test.
3. Identifying the cause of a problem may be hard.

Example from my application: We could test the Appointment class by adding a diagnostic report and checking that the appointment is updated and contains that diagnostic report.

References

1. Design Patterns in UML. *Design Patterns in UML*. Available at: https://cathyatseneca.gitbooks.io/design-patterns-in-uml/content/
2. GeeksforGeeks. *Factory method Design Pattern*. Available at: https://www.geeksforgeeks.org/factory-method-for-designing-pattern/
3. GeeksforGeeks. *Proxy Design Pattern*. Available at: https://www.geeksforgeeks.org/proxy-design-pattern/
4. GeeksforGeeks. *Chain of Responsibility Design Pattern*. Available at: https://www.geeksforgeeks.org/chain-responsibility-design-pattern/
5. Studocu. *[Solved] Discuss the differences between developerproduced and vendor. Advanced Programming (1651)*. Available at: https://www.studocu.com/row/messages/question/4324399/discuss-the-differences-between-developer-produced-and-vendor-provide-automatic-testing-tools
6. The CTO Club. *Unit Testing: Advantages & Disadvantages*. Available at: https://thectoclub.com/software-development/unit-testing/
7. BrowserStack. *What is Component Testing? (with Examples)*. Available at: https://www.browserstack.com/guide/what-is-component-testing
8. BrowserStack. *What is System Testing? (Examples, Use Cases, Types)*. Available at: https://www.browserstack.com/guide/what-is-system-testing
9. *Integration Testing - Types, Advantages and Disadvantages*. Available at: https://grotechminds.com/integration-testing-types-advantages-and-disadvantages/